

■ Lab 2 — Resource Block : EC2 & Security Group

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Compris la structure d'un **resource block** Terraform
- Déployé une **instance EC2** et un **Security Group sans règles** sur AWS
- Utilisé le workflow complet : `terraform init` → `validate` → `fmt` → `plan` → `apply` → `destroy`

■ Étape 1 — Rappel : Structure d'un Resource Block

Un resource block est composé de :

```
resource "<TYPE>" "<NOM_LOCAL>" {  
  attribut1 = "valeur1"  
  attribut2 = "valeur2"  
}
```

- **Mot-clé** `resource`
- **Type de ressource** : ex. `aws_instance`, `aws_security_group`
- **Nom local** : utilisé uniquement dans le code Terraform pour référencer la ressource
- **Attributs** : propriétés de la ressource

■ Terraform crée une ****dépendance implicite**** quand un attribut d'une ressource est référencé dans une autre.
Pas besoin de `depends_on` dans ce cas.

■ Étape 2 — Structure des fichiers

Placez-vous dans le dossier du lab :

```
cd labs/lab02-basics
```

Créez les fichiers suivants :

```
lab02-basics/  
■■■ versions.tf  
■■■ variables.tf  
■■■ main.tf  
■■■ outputs.tf
```

■ Étape 3 — Bloc `terraform` et `provider`

`versions.tf`

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "eu-west-1"
}
```

■ Étape 4 — Variable username

`variables.tf`

```
variable "username" {
  description = "Votre prenom - permet de differencier les ressources entre participants"
  type        = string
}
```

■■ Cette variable est ****obligatoire**** — elle sera demandée à chaque `terraform apply` si vous ne la fournissez pas via `-var`. Elle permet d'éviter les conflits de noms entre les 12 participants sur le même compte AWS.

■ Étape 5 — Main.tf complet

`main.tf`

```
resource "aws_security_group" "lab2_sg" {
  name          = "lab2-sg-${var.username}"
  description   = "Security group sans regles - Lab 2 - ${var.username}"

  tags = {
    Name     = "lab2-sg-${var.username}"
    Lab      = "lab2"
    Username = var.username
  }
}

resource "aws_instance" "lab2_instance" {
  ami              = "ami-0694d931cee176e7d"
  instance_type    = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab2_sg.id]

  tags = {
    Name     = "lab2-instance-${var.username}"
    Lab      = "lab2"
    Username = var.username
  }
}
```

■ `aws_security_group.lab2_sg.id` est une **dépendance implicite** — Terraform crée le Security Group avant l'instance automatiquement.

■ L'AMI `ami-0694d931cee176e7d` est une Amazon Linux 2023 valide en `eu-west-1`.

■ Étape 6 — Outputs

`outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value      = aws_instance.lab2_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value      = aws_instance.lab2_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value      = aws_security_group.lab2_sg.id
}

output "security_group_name" {
  description = "Nom du Security Group"
  value      = aws_security_group.lab2_sg.name
}
```

■ Étape 7 — Workflow Terraform

1. Initialiser

```
terraform init
```

2. Valider

```
terraform validate
```

Résultat attendu :

```
Success! The configuration is valid.
```

3. Formater

```
terraform fmt
```

4. Planifier

```
terraform plan -var="username=<votre-prenom>"
```

■ Remplacez `` par votre prénom en minuscules sans accent. Ex : `terraform plan -var="username=alice"``

5. Appliquer

```
terraform apply -var="username=<votre-prenom>"
```

Tapez **yes** pour confirmer.

6. Vérifier les outputs

```
terraform output
```

■ Étape 8 — Vérification sur AWS

```
# Remplacez <votre-prenom> par votre prénom
aws ec2 describe-instances \
  --filters "Name=tag:Username,Values=<votre-prenom>" \
  --query "Reservations[*].Instances[*].[InstanceId,State.Name,PublicIpAddress]" \
  --output table

aws ec2 describe-security-groups \
  --filters "Name=tag:Username,Values=<votre-prenom>" \
  --query "SecurityGroups[*].[GroupId,GroupName]" \
  --output table
```

■ Étape 9 — Explorer les dépendances

```
terraform graph
```

■ La sortie au format DOT montre clairement que `aws_instance.lab2_instance` dépend de `aws_security_group.lab2_sg`.

■ Étape 10 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

Tapez **yes** pour confirmer.

■ Toujours détruire les ressources à la fin d'un lab pour éviter des coûts AWS inutiles.

■ Validation du Lab

#	Critère	Validé
1	<code>`terraform init`</code> réussi, provider AWS téléchargé	■
2	<code>`terraform validate`</code> retourne "Success"	■
3	<code>`terraform plan`</code> affiche 2 ressources avec votre prénom	■
4	<code>`terraform apply`</code> crée l'instance EC2 et le SG	■
5	<code>`terraform output`</code> affiche l'instance ID et le SG	■
6	Ressources visibles avec le tag <code>`Username=`</code>	■
7	<code>`terraform destroy`</code> supprime les 2 ressources	■

■■ Résolution des problèmes courants

Problème	Cause	Solution
<code>`Reference to undeclared resource`</code>	<code>`main.tf`</code> incomplet ou vide	Vérifier que les 2 ressources sont dans <code>`main.tf`</code>
<code>`Error: No valid credential sources`</code>	Credentials AWS manquants	Relancer <code>`export AWS_ACCESS_KEY_ID=...`</code>
<code>`Error: InvalidAMIID`</code>	Mauvaise région	Vérifier que la région est <code>`eu-west-1`</code>
<code>`var.username`</code> demandé en boucle	<code>`-var`</code> oublié	Ajouter <code>`-var="username="`</code> à la commande

■ ****Fin du Lab 2 — Vous avez déployé votre première infrastructure avec Terraform !****

Le Lab 3 explore la gestion du ****State file**** : ``terraform state list``, ``show``, ``mv`` et ``refresh``.